

11. Web Frameworks and Forms

Hello!

While we wait for everyone,
feel free to share in chat what
your favorite app is!

Announcements

No sessions on Friday, have a great three-day weekend!

Frontend Technologies

HTML (Hypertext Markup Language) structures and formats layout

CSS (Cascading Style Sheets) styles content by adding design elements like colors, fonts, spacing

JavaScript (JS) creates interaction and dynamic (changing) content

SEO Tech
Developer

Web Frameworks

Imagine this

Create personal website

Website probably has many pages

- example: college page (collegename.edu known as landing page) shows general info, available majors, admissions info, college campus info

What different pages might personal website have?

- yourname.com - general info (name, major, school)
- portfolio of projects - descriptions and links to projects
- in future: page of career + what you did at each company
- blog of technical learnings
- contact page - LinkedIn, GitHub, form to email you

Website pages: URLs

Landing page - `yourname.com`

Projects - `yourname.com/projects`

Career - `yourname.com/career`

Blog - `yourname.com/blog`

Contact - `yourname.com/contact`

...

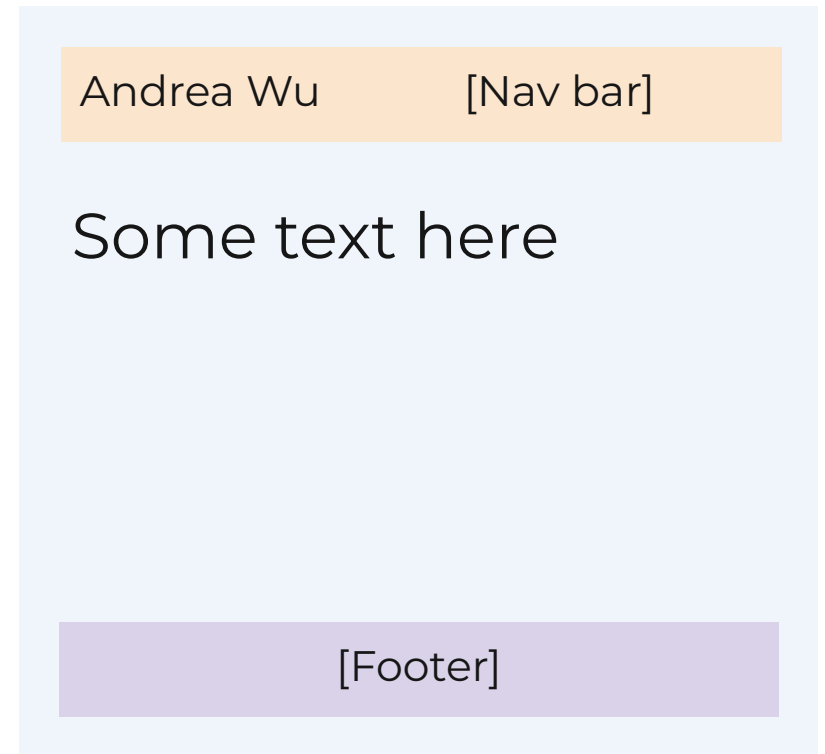
For each page, need to create HTML file - 5 HTML files in this example

Website pages

Landing page, projects, career, blog, contact, ...

Probably all have similar elements:

- same navigation bar (home, projects, career, blog, contact)
- same spacing on each page where text is
- same footer (copyright, LinkedIn, GitHub, X)
- same background color and text color / size

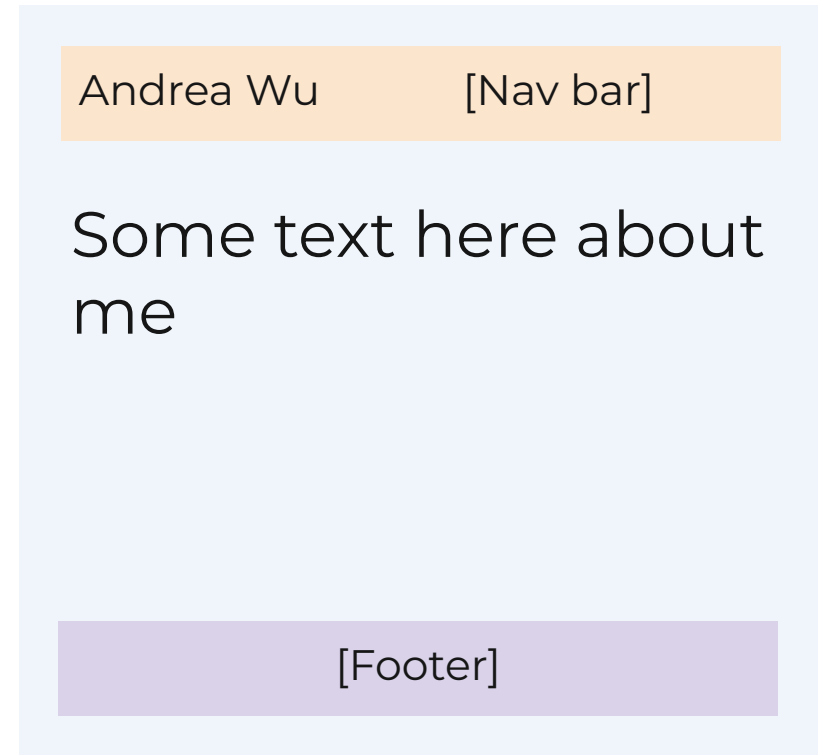


Website pages

```
<html>
  <head>
    <title>Andrea Wu</title>
  </head>
  <body>
    <div class="topnav">
      <a href="/">Andrea Wu</a>
      <a href="/projects">Projects</a>
      <a href="/career">Career</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </div>

    <p>Some text here about me</p>

    <div class="footer">
      <p>&copy; 2026 Andrea Wu</p>
      <a href="github.com/me" target="_blank">GitHub</a> |
      <a href="linkedin.com/in/me" target="_blank">LinkedIn</a> |
      <a href="x.com/me" target="_blank">X</a>
    </div>
  </body>
</html>
```



myname.com

Website pages

```
<html>
  <head>
    <title>Andrea Wu</title>
  </head>
  <body>
    <div class="topnav">
      <a href="/">Andrea Wu</a>
      <a href="/projects">Projects</a>
      <a href="/career">Career</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </div>

    <p>Some text here about my career</p>

    <div class="footer">
      <p>&copy; 2026 Andrea Wu</p>
      <a href="github.com/me" target="_blank">GitHub</a> |
      <a href="linkedin.com/in/me" target="_blank">LinkedIn</a> |
      <a href="x.com/me" target="_blank">X</a>
    </div>
  </body>
</html>
```

Andrea Wu

[Nav bar]

Some text here about
my career

[Footer]

myname.com/career

Website pages

Landing page, projects, career, blog, contact, ...

Talked about landing page and career

What do you think other pages look like - what's same / different?

- same: navigation bar, footer, background colors, text sizes
- different: text / format on content section

What about code? What's same / different?



myname.com/??

Website pages

```
<html>
  <head>
    <title>Andrea Wu</title>
  </head>
  <body>
    <div class="topnav">
      <a href="/">Andrea Wu</a>
      <a href="/projects">Projects</a>
      <a href="/career">Career</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </div>
    <p>Some text here about ??</p>
    <div class="footer">
      <p>&copy; 2026 Andrea Wu</p>
      <a href="github.com/me" target="_blank">GitHub</a> |
      <a href="linkedin.com/in/me" target="_blank">LinkedIn</a> |
      <a href="x.com/me" target="_blank">X</a>
    </div>
  </body>
</html>
```



myname.com/??

Everything is same
except content
section

Website pages

Copy / paste code each time create new page, just change content part?

What happens if:

- need to add item to navigation bar?
- Twitter changes name to X, link changes, so have to change footer?
- want to change background color? text color?
- decide to add image to every page before content?

Need to make above changes in ALL pages

Frameworks to the rescue!

Define ONE layout to use many times without copying and pasting code

If need to change, only change in ONE place

Web frameworks why #1

Remember this?

Website pages: URLs

Landing page - [yourname.com](#)

Projects - [yourname.com/projectscareer](#)

Career - [yourname.com/career](#)

Blog - [yourname.com/blog](#)

Contact - [yourname.com/contact](#)

...

Remember this?

Website pages: URLs

Landing page - `yourname.com`

Projects - `yourname.com/projectscareer`

Career - `yourname.com/career`

Blog - `yourname.com/blog`

Contact - `yourname.com/contact`

Have many HTML pages (one for each page above) - how to connect each HTML page to the proper URL?

Website pages: URLs

Many ways to do this with pure HTML and JS

How?
Lots of code

Traditional websites:

- load new HTML file for every route (website.com/**route**)
- when need HTML file for certain route, client (browser in this case) sends HTTP GET request to server
- server returns with HTTP response with HTML in body
- browser throws away current page, parses new HTML, downloads needed CSS / JS / images, loads new page

Server-Side Routing: every route = new request + new HTML document

Website pages: URLs

Many ways to do this with pure HTML and JS

Web applications:

- user clicks new route (ie /career)
- browser URL changes
- JS notices
- router decides which page to show
- DOM updates

Client-Side Routing: JS changes UI

Why is this better? Page never needs to reload

How?
Lots of code

Frameworks to the rescue!

Automatically does routing once defined

Don't need to write lots of code (requires multiple files - HTML, JS, etc)

Web frameworks why #2

Web Framework: Why?

1. Define ONE layout to use many times without copying and pasting code. If need to change, only change in ONE place
2. Routing
3. (Some other reasons - state management)

Framework: what?

Structured code simplifying creating desktop / mobile / web app

Usually includes set of libraries to perform various tasks

Web framework: framework specifically for making web apps

Examples: Django, **Flask**, Bottle, Angular

Flask

Web framework using Python

Helps with routing and fetching, HTML / CSS for layout

“Hello, world” web app is ~10 lines of code, nothing else!!

Micro-framework: abstracts very little (you can decide what to abstract)

Hello World in Flask (in .py file)

```
from flask import Flask

# Gets name of .py file so Flask knows
app = Flask(__name__)

# Tells what URL the method below is related to
@app.route("/")
def hello_world():
    # Prints HTML to webpage
    return "<p>Hello, World!</p>"
```

Automatically does routing - no need for you to write code for it!

When you go to yourURL.com/, <p>Hello, World!</p> appears

How to change above code if want to go to yourURL.com/contact?

Running Flask in Codio (option 1)

Add to end of Python file:

```
if __name__ == '__main__':  
    app.run(host="0.0.0.0")
```

Start Flask server using normal Python command:

```
python3 file_name.py
```

Stop server using CTRL + C

Click "Flask Application" in top menu bar to see webpage

- If see 404, check that server is running

Running Flask in Codio (option 2)

Set environment variable:

```
export FLASK_APP=file_name_without_extension
```

e.g. if file is `hello.py`, run `export FLASK_APP=hello`

Start Flask server by running:

```
flask run --host=0.0.0.0
```

Stop server using CTRL + C

Click "Flask Application" in top menu bar to see webpage

- If see 404, check that server is running

Adding page to website

Set URL – for example website.com/about is:

```
@app.route("/about")
```

Make function that does something on that page:

```
def about():  
    return "<p>About us!</p>"
```

View new page by:

Starting Flask server

Open website

Add **/about** to URL

Making Changes

If make change with server running and re-load webpage, change isn't there. Either:

- Re-start server - stop using CTRL+C and re-start
- Run server in DEBUG mode

To run server in DEBUG mode, change last line of code to:

- `app.run(debug=True, host="0.0.0.0")`

Web Framework: Why?

1. Define ONE layout to use many times without copying and pasting code. If need to change, only change in ONE place
2. Routing

SEO Tech
Developer

Templates

Remember this?

```
from flask import Flask

# Gets name of .py file so Flask knows
app = Flask(__name__)

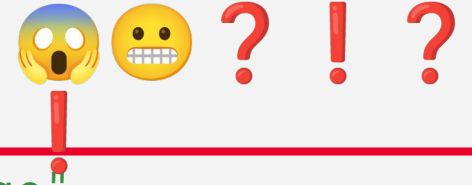
# Tells what URL the method below is related to
@app.route("/")
def hello_world():
    # Prints HTML to webpage
    return "<p>Hello, World!</p>"
```

What does it do?

How to change so that webpage shows actual informational page?

Rendering HTML

```
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return '<html itemscope="" itemtype="http://schema.org/WebPage"
lang="en"><head><meta content="Search world's information."name="description">
<meta content="noodp" name="robots"><meta content="/logos/doodles/2017/colemans-1
25th.gif" itemprop="image"><link href="/images/branding/product/ico/googleg.ico"
rel="shortcut icon"><meta content="Bessie Coleman's 125th birthday!
#GoogleDoodle" property="og:description">...'
```



Let's put that into index.html

```
<!DOCTYPE html>
<html lang="en" itemscope itemtype="http://schema.org/WebPage">
  <head>
    <meta charset="UTF-8">
    <meta name="description" content="Search world's information.">
    <meta name="robots" content="noodp">
    <meta itemprop="image" content="/logos/doodles/2017/colemans-125th.gif">
    <link rel="shortcut icon" href="/images/branding/product/ico/googleg.ico">
    <meta property="og:description"
      content="Bessie Coleman's 125th birthday! #GoogleDoodle">
    <title>Google</title>
  </head>
  <body>
  </body>
</html>
```



Rendering HTML

```
import flask
app = flask.Flask(__name__)
@app.route('/')
def index():
    return '<html itemscope="" itemtype="http://schema.org/WebPage"
lang="en"><head><meta content="Search world's information." name="description">
<meta content="noodp" name="robots"><meta content="/logos/doodles/2017/colemans-1
25th.gif" itemprop="image"><link href="/images/branding/product/ico/googleg.ico"
rel="shortcut icon"><meta content="Bessie Coleman's 125th birthday!">...'
```

Put into index.html

😱 😬 ? ! ?

```
from flask import Flask, render_template
app = flask.Flask(__name__)
@app.route('/')
def index():
    return render_template("index.html")
```

😓 😊

What is that doing?

Rendering templates

Update Python file:

1. Import `render_template`
2. Call `render_template` instead of returning raw HTML

```
from flask import Flask, render_template
app = Flask(__name__)
@app.route("/")
def home():
    return render_template('home.html')
```

Templates

In Flask, templates enable putting HTML into nicely formatted files

- No need to stuff all HTML into crazy string

Flask: built-in templating language called Jinja2

To use, **must** create `templates` folder (Flask looks for it) to hold HTML files

Let's say create 3 files:

- `layout.html`
- `home.html`
- `projects.html`

Remember this?

```
<html>
  <head>
    <title>Andrea Wu</title>
  </head>
  <body>
    <div class="topnav">
      <a href="/">Andrea Wu</a>
      <a href="/projects">Projects</a>
      <a href="/career">Career</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </div>
    <p>Some text here about ??</p>
    <div class="footer">
      <p>&copy; 2026 Andrea Wu</p>
      <a href="github.com/me" target="_blank">GitHub</a> |
      <a href="linkedin.com/in/me" target="_blank">LinkedIn</a> |
      <a href="x.com/me" target="_blank">X</a>
    </div>
  </body>
</html>
```



myname.com/??

Everything is same
except content
section

Templates: use one layout

```
<html>
  <head>
    <title>Andrea Wu</title>
  </head>
  <body>
    <div class="topnav">
      <a href="/">Andrea Wu</a>
      <a href="/projects">Projects</a>
      <a href="/career">Career</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </div>
    {% block content %} {% endblock %}
    <div class="footer">
      <p>&copy; 2026 Andrea Wu</p>
      <a href="github.com/me" target="_blank">GitHub</a> |
      <a href="linkedin.com/in/me" target="_blank">LinkedIn</a> |
      <a href="x.com/me" target="_blank">X</a>
    </div>
  </body>
</html>
```

Section for different pages

This page is layout.html

All other pages use layout.html

How?

Use layout.html

```
{% extends "layout.html" %}

{% block content %}

    <p>I can add here whatever is unique to my page!</p>

{% endblock %}
```

Add code to any HTML file; Jinja auto-includes all text from layout.html

What code means:

1. `{% extends "layout.html" %}` - inherit code from layout.html
2. `{% block content %}...{% endblock content %}` - specifies part of layout.html template to fill in

Templates: use one layout

```
<html>
  <head>
    <title>Andrea Wu</title>
  </head>
  <body>
    <div class="topnav">
      <a href="/">Andrea Wu</a>
      <a href="/projects">Projects</a>
      <a href="/career">Career</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </div>
    {% block content %} {% endblock %}
    <div class="footer">
      <p>&copy; 2026 Andrea Wu</p>
      <a href="github.com/me" target="_blank">GitHub</a> |
      <a href="linkedin.com/in/me" target="_blank">LI</a> |
      <a href="x.com/me" target="_blank">X</a>
    </div>
  </body>
</html>
```

layout.html

```
{% extends "layout.html" %}
```

```
{% block content %}
```

```
  <p>This is content for Projects page!</p>
```

```
{% endblock %}
```

aPythonFile.py

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
@app.route("/projects")
```

```
def projects():
```

```
    return render_template('projects.html')
```

Use layout.html

```
{% extends "layout.html" %}  
{% block content %}  
    <p>I can add here whatever is unique to my page!</p>  
{% endblock %}
```

What if want to add same element to every page but just change text?

```
{% extends "layout.html" %}  
{% block content %}  
    <h2>{{subtitle}}</h2>  
    <p>I can add here whatever is unique to my page!</p>  
{% endblock %}
```


Use layout.html

```
{% extends "layout.html" %}
{% block content %}
    <h2>{{subtitle}}</h2>
    <p>I can add here whatever is unique to my page!</p>
{% endblock %}
```

Use variable as placeholder, then give variable a value when using template

```
from flask import Flask, render_template
app = Flask(__name__)
@app.route("/projects")
def projects():
    return render_template('projects.html', subtitle='Projects')
```

SEO Tech
Developer

Forms

Forms: what?

Python library to validate and render **forms**: made of **Fields** with **Validators**

Fields include form field types where user inputs information:

- **StringField** – field to input text
- **PasswordField** – like StringField but value is not rendered to browser
- **SubmitField** - check if given submit button was pressed

Validators check user input

- **DataRequired** - sets required flag on fields it is used on
- **Length** - validates string length
- **Email** - validates email address using `email_validator` package
- **EqualTo** – compares values of two fields - used to facilitate password change form

Flask-WTF: library that integrated WTForms with Flask

Create Forms.py

```
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Length, Email, EqualTo

class RegistrationForm(FlaskForm):
    username = StringField('Username',
                           validators=[DataRequired(), Length(min=2, max=20)])
    email = StringField('Email', validators=[DataRequired(), Email()])
    password = PasswordField('Password', validators=[DataRequired()])
    confirm_password = PasswordField('Confirm Password',
                                     validators=[DataRequired(), EqualTo('password')])
    submit = SubmitField('Sign Up')
```

Add Form to Website (hello.py)

Make sure Flask app can see form by importing in `hello.py`:

```
from forms import RegistrationForm
```

Add new page to site:

```
@app.route("/register", methods=['GET', 'POST'])
def register():
    form = RegistrationForm()
    return render_template('register.html', title='Register', form=form)
```

Example register.html

```
{% extends "layout.html" %}
{% block content %}
  <div class="content-section">
    <form method="POST" action=""> <!--sends information to form -->
      {{ form.hidden_tag() }} <!-- passes your secret key ** -->
      <fieldset class="form-group">
        <legend class="border-bottom mb-4">Join Today</legend>
        <div class="form-group"> <!--add username field -->
          {{ form.username.label(class="form-control-label") }}
          {{ form.username(class="form-control form-control-lg") }}
        </div>
        <div class="form-group"> <!--add email field -->
          {{ form.email.label(class="form-control-label") }}
          {{ form.email(class="form-control form-control-lg") }}
        </div>
        <div class="form-group"> <!--add password field -->
          {{ form.password.label(class="form-control-label") }}
          {{ form.password(class="form-control form-control-lg") }}
        </div>
        <div class="form-group"> <!--add confirm password field -->
          {{ form.confirm_password.label(class="form-control-label") }}
          {{ form.confirm_password(class="form-control form-control-lg") }}
        </div>
      </fieldset>
      <div class="form-group">{{ form.submit(class="btn btn-outline-info") }} </div> <!--add button field -->
    </form>
  </div>
{% endblock content %}
```

Secrets Keys?

To prevent malicious sites from executing unwanted actions on forms (ie steal data), need secret key:

1. Open Python interpreter and generate 16-byte token - copy without quotes

```
>>> import secrets  
>>> secrets.token_hex(16)
```

2. In main .py file, after `app = Flask(__name__)` add...
`app.config['SECRET_KEY'] = 'the key you generated'`

Passing Success Message

After instantiate form (form = RegistrationForm()) in main.py file:

```
if form.validate_on_submit():  
    flash(f'Account created for {form.username.data}!', 'success')  
    return redirect(url_for('home'))
```

f-string lets us insert variable - ie data filled in by user in username field

`success` tells bootstrap type of message sent so can style accordingly

Form successfully validated, account "created" - send user to home screen

To re-direct properly in Codio, add:

```
from flask import Flask, render_template, url_for, flash, redirect  
from flask_behind_proxy import FlaskBehindProxy  
app = Flask(__name__)  
proxied = FlaskBehindProxy(app)
```


Passing Success Message

```
<!-- layout.html -->
```

```
...
```

```
<h1>This shows up on every page</h1>
```

```
{% with messages = get_flashed_messages(with_categories=true) %}  
  {% if messages %}  
    {% for category, message in messages %}  
      <div class="alert alert-{{ category }}">  
        {{ message }}  
      </div>  
    {% endfor %}  
  {% endif %}  
{% endwith %}  
  
{% block content %}{% endblock %}
```

Showing Validation Errors

```
{% extends "layout.html" %}
{% block content %}
<div class="content-section">
  <form method="POST" action="">
    {{ form.hidden_tag() }}
    <fieldset class="form-group">
      <div class="form-group">
        {{ form.username.label(class="form-control-label") }}
        {% if form.username.errors %}
          {{ form.username(class="form-control form-control-lg is-invalid") }}
          <div class="invalid-feedback">
            {% for error in form.username.errors %}
              <span>{{ error }}</span>
            {% endfor %}
          </div>
        {% else %}
          {{ form.username(class="form-control form-control-lg") }}
        {% endif %}
      </div>
    </fieldset>
  </form>
</div>
{% endblock content %}
```

Passes data validation errors to user

SEO Tech
Developer

Summary

Web Frameworks

What?

Structured code **simplifying** creating web app

Usually includes set of **libraries** to perform various tasks

Why?

Define **one layout** to use many times without copying and pasting code. If need to change, only change in one place

Routing

(Some other reasons - state management)

SEO Tech
Developer

Thank you!